

# Systems - Notes

Prof: Mr. Joel Swanson

office: 313 Anne Belk Hall

email: [swansonja@appstate.edu](mailto:swansonja@appstate.edu)

Office hours: 10:00 AM to 10:50 AM and 1:00 PM to 1:50 PM. Email me to set up a Zoom appointment if you prefer.

## Things

## Grades

### Online Assignments

Online quizzes, homework, and programming assignments will be posted on AsULearn. Unless otherwise specified, all assignments are due before the start of the class on the due date. No assignment will be accepted once the solution is discussed in the classroom.

I will announce assignments and due dates on the message board. The due date and time in the class [announcements](#) are the official due date and time unless a subsequent post changes the due date or time. If you notice a discrepancy between an online assignment due date and the one posted on the forum, the forum will be considered the correct date and time.

I suggest you use some calendaring system to keep track of the assignments and do not rely on the AsULearn notification.

### Programming Assignments and Homework

Programming is a large part of this course. There will be programming in C and in LC3 assembly and machine language.

Homework is generally smaller programming tasks and contribute less to your overall grade. Assignments are larger, more complex sets of tasks that are tied together and count for a much larger part of your grade.

IF YOU MAKE LESS THAN 50% ON THE PROGRAMMING ASSIGNMENTS AVERAGE YOU FAIL THE COURSE.

## Quizzes

There will be in class quizzes and online quizzes. **The lowest quiz grade will be dropped.** Quizzes are used to ensure in-class participation and understanding. In class quizzes are part

of the lecture process and therefore **cannot** be made up. In the event of an excused absence, students will be given credit for the in-class quiz.

## Exams

Exams will be announced at least five days in advance and may be cumulative. Any student missing an exam for an unexcused absence will receive a zero on the exam. A student missing an exam for an excused absence when the absence is known in advance **MUST** plan to take the exam early. Students missing an exam for an emergency excused absence where there was not an ability to take the exam early, will be allowed to substitute their final exam grade for the missing exam.

## Final Exam

The final exam will be held in accordance with the final exam schedule as listed on the official university web page. Makeup final exams will only be given in extreme circumstances, such as student hospitalization or death in the family. Both of which must be reported through student services. The final exam will be given on the date and time specified by the registrar's exam schedule web page. A link to the registrar's final exam schedule web page is on AsULearn.

**IF YOU FAIL THE FINAL EXAM WITH A SCORE LESS THAN 50%, YOU FAIL THE COURSE.**

## Curves

Exam grades, and only exam grades, will occasionally be curved. If the average of an exam is below 70%, I may curve the exam by whole points until an average of 69.5% to 70.5% is achieved. This curve cannot exceed 10 points regardless of average. Curved grades that exceed 100% will be cut off at 100%.

## Late Work

No late work will be accepted except under the most extreme circumstances. Most assignments are open for multiple days. Unless you are hospitalized for the **entire** assignment period you will not be given extra time.

**Do not plan to do your work the day before the assignment is due. That is bad planning on your part and bad planning is the real reason assignments are missed.** An emergency on that last day means you will not have time to complete the assignment.

Plan to do your work the day it is assigned or the day after it is assigned and that way if an emergency happens, you will have plenty of time to deal with the emergency and get your work done. This is good planning and is how you should approach assignments as an adult.

## Grading

- [Attendance](#): 10%
- Programming Assignments: 30%
- Homework and Quizzes: 10%
- Midterm Exam: 20%
- Final Exam (cumulative): 30%

## [Attendance](#)

- [Attendance](#) will be taken every class period.
- **If you arrive AFTER the roll has been taken, you are absent.**
- If the [attendance](#) is done electronically, it is your responsibility to make sure you have registered every day of class.
- If your name is not on the [attendance](#) record it is your fault and it will not be changed. If you have an issue in class, you MUST tell me at that time or you will be counted absent.
- Six or more unexcused absences will result in failure of the course

## Communications

Assignments and assignment updates will occasionally be posted to the AsULearn forum. I will post these notifications to the AsULearn forum AND will send an email to your designated account. I expect you to receive and check your official campus email with some regularity. While I would never post an assignment and expect it to be due the next day, I will, and do, expect you to check your email over the course of a weekend. If I inform you in class that an assignment will be posted, I expect you to check for that assignment online even without an email.

If class is cancelled due to weather, there will be an online lecture or online assignment. You are expected to check ASULearn.

## Notes

```
java -jar LC3sim.jar
```

| Dec | Hx | Oct | Char                               | Dec | Hx | Oct | Html  | Chr          | Dec | Hx | Oct | Html  | Chr      | Dec | Hx | Oct | Html   | Chr        |
|-----|----|-----|------------------------------------|-----|----|-----|-------|--------------|-----|----|-----|-------|----------|-----|----|-----|--------|------------|
| 0   | 0  | 000 | <b>NUL</b> (null)                  | 32  | 20 | 040 | &#32; | <b>Space</b> | 64  | 40 | 100 | &#64; | <b>@</b> | 96  | 60 | 140 | &#96;  | <b>`</b>   |
| 1   | 1  | 001 | <b>SOH</b> (start of heading)      | 33  | 21 | 041 | &#33; | <b>!</b>     | 65  | 41 | 101 | &#65; | <b>A</b> | 97  | 61 | 141 | &#97;  | <b>a</b>   |
| 2   | 2  | 002 | <b>STX</b> (start of text)         | 34  | 22 | 042 | &#34; | <b>"</b>     | 66  | 42 | 102 | &#66; | <b>B</b> | 98  | 62 | 142 | &#98;  | <b>b</b>   |
| 3   | 3  | 003 | <b>ETX</b> (end of text)           | 35  | 23 | 043 | &#35; | <b>#</b>     | 67  | 43 | 103 | &#67; | <b>C</b> | 99  | 63 | 143 | &#99;  | <b>c</b>   |
| 4   | 4  | 004 | <b>EOT</b> (end of transmission)   | 36  | 24 | 044 | &#36; | <b>\$</b>    | 68  | 44 | 104 | &#68; | <b>D</b> | 100 | 64 | 144 | &#100; | <b>d</b>   |
| 5   | 5  | 005 | <b>ENQ</b> (enquiry)               | 37  | 25 | 045 | &#37; | <b>%</b>     | 69  | 45 | 105 | &#69; | <b>E</b> | 101 | 65 | 145 | &#101; | <b>e</b>   |
| 6   | 6  | 006 | <b>ACK</b> (acknowledge)           | 38  | 26 | 046 | &#38; | <b>&amp;</b> | 70  | 46 | 106 | &#70; | <b>F</b> | 102 | 66 | 146 | &#102; | <b>f</b>   |
| 7   | 7  | 007 | <b>BEL</b> (bell)                  | 39  | 27 | 047 | &#39; | <b>'</b>     | 71  | 47 | 107 | &#71; | <b>G</b> | 103 | 67 | 147 | &#103; | <b>g</b>   |
| 8   | 8  | 010 | <b>BS</b> (backspace)              | 40  | 28 | 050 | &#40; | <b>(</b>     | 72  | 48 | 110 | &#72; | <b>H</b> | 104 | 68 | 150 | &#104; | <b>h</b>   |
| 9   | 9  | 011 | <b>TAB</b> (horizontal tab)        | 41  | 29 | 051 | &#41; | <b>)</b>     | 73  | 49 | 111 | &#73; | <b>I</b> | 105 | 69 | 151 | &#105; | <b>i</b>   |
| 10  | A  | 012 | <b>LF</b> (NL line feed, new line) | 42  | 2A | 052 | &#42; | <b>*</b>     | 74  | 4A | 112 | &#74; | <b>J</b> | 106 | 6A | 152 | &#106; | <b>j</b>   |
| 11  | B  | 013 | <b>VT</b> (vertical tab)           | 43  | 2B | 053 | &#43; | <b>+</b>     | 75  | 4B | 113 | &#75; | <b>K</b> | 107 | 6B | 153 | &#107; | <b>k</b>   |
| 12  | C  | 014 | <b>FF</b> (NP form feed, new page) | 44  | 2C | 054 | &#44; | <b>,</b>     | 76  | 4C | 114 | &#76; | <b>L</b> | 108 | 6C | 154 | &#108; | <b>l</b>   |
| 13  | D  | 015 | <b>CR</b> (carriage return)        | 45  | 2D | 055 | &#45; | <b>-</b>     | 77  | 4D | 115 | &#77; | <b>M</b> | 109 | 6D | 155 | &#109; | <b>m</b>   |
| 14  | E  | 016 | <b>SO</b> (shift out)              | 46  | 2E | 056 | &#46; | <b>.</b>     | 78  | 4E | 116 | &#78; | <b>N</b> | 110 | 6E | 156 | &#110; | <b>n</b>   |
| 15  | F  | 017 | <b>SI</b> (shift in)               | 47  | 2F | 057 | &#47; | <b>/</b>     | 79  | 4F | 117 | &#79; | <b>O</b> | 111 | 6F | 157 | &#111; | <b>o</b>   |
| 16  | 10 | 020 | <b>DLE</b> (data link escape)      | 48  | 30 | 060 | &#48; | <b>0</b>     | 80  | 50 | 120 | &#80; | <b>P</b> | 112 | 70 | 160 | &#112; | <b>p</b>   |
| 17  | 11 | 021 | <b>DC1</b> (device control 1)      | 49  | 31 | 061 | &#49; | <b>1</b>     | 81  | 51 | 121 | &#81; | <b>Q</b> | 113 | 71 | 161 | &#113; | <b>q</b>   |
| 18  | 12 | 022 | <b>DC2</b> (device control 2)      | 50  | 32 | 062 | &#50; | <b>2</b>     | 82  | 52 | 122 | &#82; | <b>R</b> | 114 | 72 | 162 | &#114; | <b>r</b>   |
| 19  | 13 | 023 | <b>DC3</b> (device control 3)      | 51  | 33 | 063 | &#51; | <b>3</b>     | 83  | 53 | 123 | &#83; | <b>S</b> | 115 | 73 | 163 | &#115; | <b>s</b>   |
| 20  | 14 | 024 | <b>DC4</b> (device control 4)      | 52  | 34 | 064 | &#52; | <b>4</b>     | 84  | 54 | 124 | &#84; | <b>T</b> | 116 | 74 | 164 | &#116; | <b>t</b>   |
| 21  | 15 | 025 | <b>NAK</b> (negative acknowledge)  | 53  | 35 | 065 | &#53; | <b>5</b>     | 85  | 55 | 125 | &#85; | <b>U</b> | 117 | 75 | 165 | &#117; | <b>u</b>   |
| 22  | 16 | 026 | <b>SYN</b> (synchronous idle)      | 54  | 36 | 066 | &#54; | <b>6</b>     | 86  | 56 | 126 | &#86; | <b>V</b> | 118 | 76 | 166 | &#118; | <b>v</b>   |
| 23  | 17 | 027 | <b>ETB</b> (end of trans. block)   | 55  | 37 | 067 | &#55; | <b>7</b>     | 87  | 57 | 127 | &#87; | <b>W</b> | 119 | 77 | 167 | &#119; | <b>w</b>   |
| 24  | 18 | 030 | <b>CAN</b> (cancel)                | 56  | 38 | 070 | &#56; | <b>8</b>     | 88  | 58 | 130 | &#88; | <b>X</b> | 120 | 78 | 170 | &#120; | <b>x</b>   |
| 25  | 19 | 031 | <b>EM</b> (end of medium)          | 57  | 39 | 071 | &#57; | <b>9</b>     | 89  | 59 | 131 | &#89; | <b>Y</b> | 121 | 79 | 171 | &#121; | <b>y</b>   |
| 26  | 1A | 032 | <b>SUB</b> (substitute)            | 58  | 3A | 072 | &#58; | <b>:</b>     | 90  | 5A | 132 | &#90; | <b>Z</b> | 122 | 7A | 172 | &#122; | <b>z</b>   |
| 27  | 1B | 033 | <b>ESC</b> (escape)                | 59  | 3B | 073 | &#59; | <b>;</b>     | 91  | 5B | 133 | &#91; | <b>[</b> | 123 | 7B | 173 | &#123; | <b>{</b>   |
| 28  | 1C | 034 | <b>FS</b> (file separator)         | 60  | 3C | 074 | &#60; | <b>&lt;</b>  | 92  | 5C | 134 | &#92; | <b>\</b> | 124 | 7C | 174 | &#124; | <b> </b>   |
| 29  | 1D | 035 | <b>GS</b> (group separator)        | 61  | 3D | 075 | &#61; | <b>=</b>     | 93  | 5D | 135 | &#93; | <b>]</b> | 125 | 7D | 175 | &#125; | <b>}</b>   |
| 30  | 1E | 036 | <b>RS</b> (record separator)       | 62  | 3E | 076 | &#62; | <b>&gt;</b>  | 94  | 5E | 136 | &#94; | <b>^</b> | 126 | 7E | 176 | &#126; | <b>~</b>   |
| 31  | 1F | 037 | <b>US</b> (unit separator)         | 63  | 3F | 077 | &#63; | <b>?</b>     | 95  | 5F | 137 | &#95; | <b>_</b> | 127 | 7F | 177 | &#127; | <b>DEL</b> |

Source: [www.LookupTables.com](http://www.LookupTables.com)

## Decimal - Binary - Octal - Hex – ASCII Conversion Chart

| Decimal | Binary   | Octal | Hex | ASCII | Decimal | Binary   | Octal | Hex | ASCII | Decimal | Binary   | Octal | Hex | ASCII | Decimal | Binary   | Octal | Hex | ASCII |
|---------|----------|-------|-----|-------|---------|----------|-------|-----|-------|---------|----------|-------|-----|-------|---------|----------|-------|-----|-------|
| 0       | 00000000 | 000   | 00  | NUL   | 32      | 00100000 | 040   | 20  | SP    | 64      | 01000000 | 100   | 40  | @     | 96      | 01100000 | 140   | 60  | `     |
| 1       | 00000001 | 001   | 01  | SOH   | 33      | 00100001 | 041   | 21  | !     | 65      | 01000001 | 101   | 41  | A     | 97      | 01100001 | 141   | 61  | a     |
| 2       | 00000010 | 002   | 02  | STX   | 34      | 00100010 | 042   | 22  | "     | 66      | 01000010 | 102   | 42  | B     | 98      | 01100010 | 142   | 62  | b     |
| 3       | 00000011 | 003   | 03  | ETX   | 35      | 00100011 | 043   | 23  | #     | 67      | 01000011 | 103   | 43  | C     | 99      | 01100011 | 143   | 63  | c     |
| 4       | 00000100 | 004   | 04  | EOT   | 36      | 00100100 | 044   | 24  | \$    | 68      | 01000100 | 104   | 44  | D     | 100     | 01100100 | 144   | 64  | d     |
| 5       | 00000101 | 005   | 05  | ENQ   | 37      | 00100101 | 045   | 25  | %     | 69      | 01000101 | 105   | 45  | E     | 101     | 01100101 | 145   | 65  | e     |
| 6       | 00000110 | 006   | 06  | ACK   | 38      | 00100110 | 046   | 26  | &     | 70      | 01000110 | 106   | 46  | F     | 102     | 01100110 | 146   | 66  | f     |
| 7       | 00000111 | 007   | 07  | BEL   | 39      | 00100111 | 047   | 27  | '     | 71      | 01000111 | 107   | 47  | G     | 103     | 01100111 | 147   | 67  | g     |
| 8       | 00001000 | 010   | 08  | BS    | 40      | 00101000 | 050   | 28  | (     | 72      | 01001000 | 110   | 48  | H     | 104     | 01101000 | 150   | 68  | h     |
| 9       | 00001001 | 011   | 09  | HT    | 41      | 00101001 | 051   | 29  | )     | 73      | 01001001 | 111   | 49  | I     | 105     | 01101001 | 151   | 69  | i     |
| 10      | 00001010 | 012   | 0A  | LF    | 42      | 00101010 | 052   | 2A  | *     | 74      | 01001010 | 112   | 4A  | J     | 106     | 01101010 | 152   | 6A  | j     |
| 11      | 00001011 | 013   | 0B  | VT    | 43      | 00101011 | 053   | 2B  | +     | 75      | 01001011 | 113   | 4B  | K     | 107     | 01101011 | 153   | 6B  | k     |
| 12      | 00001100 | 014   | 0C  | FF    | 44      | 00101100 | 054   | 2C  | ,     | 76      | 01001100 | 114   | 4C  | L     | 108     | 01101100 | 154   | 6C  | l     |
| 13      | 00001101 | 015   | 0D  | CR    | 45      | 00101101 | 055   | 2D  | -     | 77      | 01001101 | 115   | 4D  | M     | 109     | 01101101 | 155   | 6D  | m     |
| 14      | 00001110 | 016   | 0E  | SO    | 46      | 00101110 | 056   | 2E  | .     | 78      | 01001110 | 116   | 4E  | N     | 110     | 01101110 | 156   | 6E  | n     |
| 15      | 00001111 | 017   | 0F  | SI    | 47      | 00101111 | 057   | 2F  | /     | 79      | 01001111 | 117   | 4F  | O     | 111     | 01101111 | 157   | 6F  | o     |
| 16      | 00010000 | 020   | 10  | DLE   | 48      | 00110000 | 060   | 30  | 0     | 80      | 01010000 | 120   | 50  | P     | 112     | 01110000 | 160   | 70  | p     |
| 17      | 00010001 | 021   | 11  | DC1   | 49      | 00110001 | 061   | 31  | 1     | 81      | 01010001 | 121   | 51  | Q     | 113     | 01110001 | 161   | 71  | q     |
| 18      | 00010010 | 022   | 12  | DC2   | 50      | 00110010 | 062   | 32  | 2     | 82      | 01010010 | 122   | 52  | R     | 114     | 01110010 | 162   | 72  | r     |
| 19      | 00010011 | 023   | 13  | DC3   | 51      | 00110011 | 063   | 33  | 3     | 83      | 01010011 | 123   | 53  | S     | 115     | 01110011 | 163   | 73  | s     |
| 20      | 00010100 | 024   | 14  | DC4   | 52      | 00110100 | 064   | 34  | 4     | 84      | 01010100 | 124   | 54  | T     | 116     | 01110100 | 164   | 74  | t     |
| 21      | 00010101 | 025   | 15  | NAK   | 53      | 00110101 | 065   | 35  | 5     | 85      | 01010101 | 125   | 55  | U     | 117     | 01110101 | 165   | 75  | u     |
| 22      | 00010110 | 026   | 16  | SYN   | 54      | 00110110 | 066   | 36  | 6     | 86      | 01010110 | 126   | 56  | V     | 118     | 01110110 | 166   | 76  | v     |
| 23      | 00010111 | 027   | 17  | ETB   | 55      | 00110111 | 067   | 37  | 7     | 87      | 01010111 | 127   | 57  | W     | 119     | 01110111 | 167   | 77  | w     |
| 24      | 00011000 | 030   | 18  | CAN   | 56      | 00111000 | 070   | 38  | 8     | 88      | 01011000 | 130   | 58  | X     | 120     | 01111000 | 170   | 78  | x     |
| 25      | 00011001 | 031   | 19  | EM    | 57      | 00111001 | 071   | 39  | 9     | 89      | 01011001 | 131   | 59  | Y     | 121     | 01111001 | 171   | 79  | y     |
| 26      | 00011010 | 032   | 1A  | SUB   | 58      | 00111010 | 072   | 3A  | :     | 90      | 01011010 | 132   | 5A  | Z     | 122     | 01111010 | 172   | 7A  | z     |
| 27      | 00011011 | 033   | 1B  | ESC   | 59      | 00111011 | 073   | 3B  | ;     | 91      | 01011011 | 133   | 5B  | [     | 123     | 01111011 | 173   | 7B  | {     |
| 28      | 00011100 | 034   | 1C  | FS    | 60      | 00111100 | 074   | 3C  | <     | 92      | 01011100 | 134   | 5C  | \     | 124     | 01111100 | 174   | 7C  |       |
| 29      | 00011101 | 035   | 1D  | GS    | 61      | 00111101 | 075   | 3D  | =     | 93      | 01011101 | 135   | 5D  | ]     | 125     | 01111101 | 175   | 7D  | }     |
| 30      | 00011110 | 036   | 1E  | RS    | 62      | 00111110 | 076   | 3E  | >     | 94      | 01011110 | 136   | 5E  | ^     | 126     | 01111110 | 176   | 7E  | ~     |
| 31      | 00011111 | 037   | 1F  | US    | 63      | 00111111 | 077   | 3F  | ?     | 95      | 01011111 | 137   | 5F  | _     | 127     | 01111111 | 177   | 7F  | DEL   |

This work is licensed under the Creative Commons Attribution-ShareAlike License. To view a copy of this license, visit <http://creativecommons.org/licenses/by-sa/3.0/>

ASCII Conversion Chart.doc Copyright © 2008, 2012 Donald Weiman 22 March 2012

## LC3 Programming.

### Instructions

An instruction has two parts:

- Opcode: the operation being performed
  - Operate, Data Movement, Control
  - LC3 has 15 opcodes (consult the chart)
- Operands: the data to use in the operation
  - Data is read from somewhere as specified by the instruction and the addressing mode.
  - 5 modes: Immediate, register, PC-relative, Indirect, Base +Offset

All instructions are 16 bits.

There are no subtractions! If you want to subtract, Add a negative! How do you make something negative? Invert it with not! It's possible but not exactly as straightforward as higher level languages.

Add, and and not are the only things that change the data.

This is possibly an LC3 program in assembly -

```

; Basic lc3 program.

; Semi-colons are used to create comments.

.ORIG x3000 ; Program begins at x3000.

    LEA R0, prompt1
    PUTS

    HALT ; The program is done, all objectives met.

prompt1 .STRINGZ "Hello World!" ; String variable.

.END

```

### Add\_Two.asm

```

; Filename: ADD_TWO.asm
; Author: [Your Name]
; Purpose: To add two numbers and store the result.

    .ORIG x3000          ; Program starts at memory address x3000

MAIN
    LD R0, NUM1          ; Load the value from NUM1 into Register R0
    LD R1, NUM2          ; Load the value from NUM2 into Register R1
    ADD R2, R0, R1        ; Add the values in R0 and R1, store result in R2
    ST R2, SUM            ; Store the result from R2 into memory location SUM

    HALT                 ; Halt the program execution

; --- Data Section ---
NUM1    .FILL #25        ; Define a memory word named NUM1 with decimal value
25
NUM2    .FILL #17        ; Define a memory word named NUM2 with decimal value
17
SUM     .BLKW #1         ; Reserve one memory word for SUM (initialized to 0)

    .END                 ; End of the assembly source file

```

### Operate Instructinos

With only ADD, AND, NOT...

R1 = 5, R2 = 3, R3 = 0

Perform the following(written in hex)

R1 <-R2-R1 (Subtraction)

1. Invert R1
2. Add 1 to R1 (now R1 is -5)
3. Add R2 and R1

```
; add R1, R2, R3
NOT R1, R1
; R1 becomes Not(5) = xFFFA
ADD R1, R1, #1
; R1 becomes -5 = xFFFB
ADD R1, R2, R1
; R1 = 3 + (-5) = -2 (xFFFE)
```

R1 <-R2 | R1 (bitwise or)

1. Invert R1
2. Invert R2 (using R3 as temp workspace)
3. AND the results
4. Invert the final result

```
NOT R1, R1 ; R1 = NOT(5)
NOT R3, R2 ; R3 = NOT(3)
AND R1, R1, R3
; R1 = NOT(5) AND NOT(3)
NOT R1, R1
; R1 = NOT(NOT(5)) AND NOT(3)) = 5 | 3 = 7
```

R1 <-R2 (Transfer)

```
ADD R1, R2, #0 ; R1 = R2 + 0
```

R1 <- 0 (clear)

```
AND R1, R1, #0 ; R1 = R1 AND 0
```

Addressing Modes

- PC-Relative

- Indirect
- Base + Offset

## Loading and storing data

- PC Relative

```
LD R2, (relative distance between two addresses)
```

```
LD R2, PCOFFSET9
```

or

```
LD R2, 125
```

## Example

Assume this LD is stored at x3000, what is the effective address?

```
1111 119 876543210
5432 10
opcode|DR/SR| pc offset 2's comp value added to pc
0010 010 010101111
LD    R2    x0AF
```

3001

+0AF - offset

30B0 - result

Must be within 255 memory locations, due to how many values you get in pcoffset9 (no more room to store things)

Must keep program steps and data separated! Gotta keep 'em separated...

- Indirect
- Base + offset

## Example : Add 3 Numbers

1. Start program at x3000
2. Values 5, 7, and 17 are stored just after the halt (halt, 5, 7, 17)
3. The sum will be stored just after the above 3 values
4. Only use PC-Relative addressing (LD 0010 and ST 0011)



## 5. Will also need add (0001) (which add? Register or immediate)

```
LD R0, offset 6
LD R1, offset 6
LD R2, offset 6
ADD R3, R0, R1
ADD R3, R2, R3
ST R3, offset 4
HALT
(data pt 1 here, originally at offset 6 (6 mem locations) from start)
(data pt 2 here)
(data pt 3 here)
(final value gets stored at offset 4 from halt)
```

Turning the above into Hexadecimal (binary first, then hex)

```
0010 000 0 0000 0110 ; LD R0 pcoffset 6 (2's complement)
0010 001 0 0000 0110
0010 010 0 0000 0110
0001 011 000 000001 ; ADD R3, R0, R1
0001 011 010 000 011 ; ADD R3, R2, R3
0011 0110 0000 0100 ; ST R3, OFFset 4
1111 0000 0010 0101 ; halt
turns into...
2006 ; LD R0 OFFSET 6
2206 ; LD R1 OFFSET 6
2406 ; LD R2 OFFSET 6
1601 ; ADD R3, R0, R1
1603 ; ADD R3, R2, R3
3604 ; ST R3, OFFSET 4
F025 ; HALT
(data goes here in the program)
```

## LC3 stuff

### Branching

Branch (Br) - changes execution order if conditions are met. Detects negative (n) positive (p) or zero (z)

Works in conjunction with one of the following instructions

- ADD, AND, NOT, LD, LDI, LDR, LEA

Branch if the N is set or the P is set. Do not branch if the Z is set.

How does it branch? It takes the 2's comp PC Offset value and adds it to a register ??

check this

Positive, will skip over next bits of code. Ex: If Statement (if)(code to skip)(code to execute)

Negative, will loop back over bits of code Ex: For(until you hit branch) (go back to statement)

R2-R1 (if (r2>r1)) pseudocode

- R2>R1 Positive
- Equal Zero
- R2<R1 Negative

"If the condition is false, jump over the block of code"

```
N0T R3, R1 ; R3 <- R1
ADD R3, R3, 1 ;
ADD R3, R2, R3 ;
BR NZ , NEXT REGISTER
```

```
N0T R3, R1
ADD R3, R3, 1
ADD R3, R2, R3
BR NZ , NEXT REGISTER
```

Not, Add 1, Add values, Branch

If(condition)Else

```
if(r1>r2) //
{
    r1 = r1-r2 // block 1
}
else // BR NZP
{
    r1 = r2 - r1 // blocok 2
}
```

```
N0T R3, R2 ; R3 <-R2
ADD R3, R3, 1
ADD R3, R1, R3 ; R3 <R1-R2
BR, NZ, NEXT (2 TO BLOCK 2)
ADD R1, R3, #0 ; BLOCK 1
```

```
BR, NZP, NEXT (2 TO HALT)
NOT R1, R3 ; Inverse of r1-r2 is r2-r1 ; BLOCK 2
ADD R1, R1, #1 ; BLOCK 2
HALT
```

Rule of thumb for assembly - comment for every line of code (easier to troubleshoot)

ST

PC ST R3 (78)

IR AND R3 R1

R0 23

R1 10

R2 30

R3 19

R4 27

R5 15

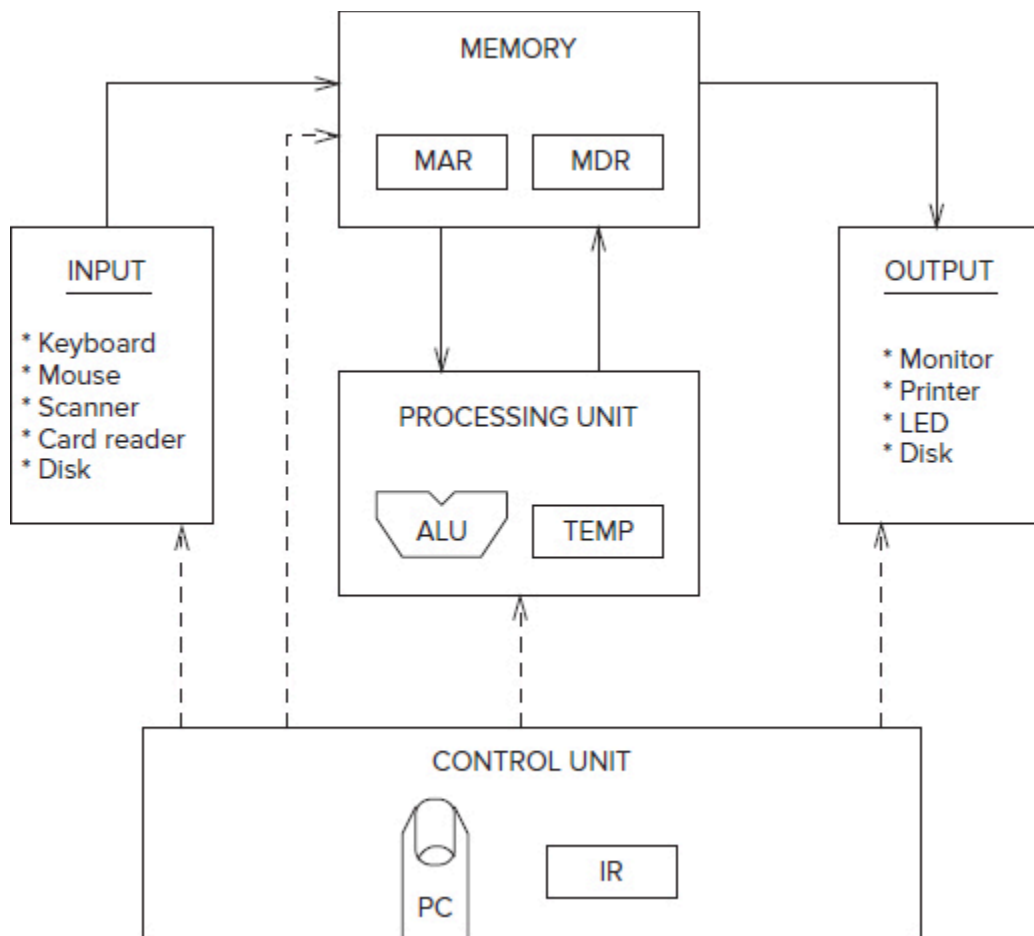
R6 14

R7 23

CC ??

## Instruction Model

The John von Neumann computer model describes a fundamental model of a computer for processing computer programs.



## Instructions

The most basic unit of computer processing is the instruction. It is made up of the opcode and the operand. Instructions are processed under the direction of the control unit in a very systematic, step-by-step manner.

The instruction cycle

1. Fetch
2. Decode
3. Evaluate Address
4. Fetch Operands
5. Execute
6. Store Result

### Fetch

Obtain the next instruction from memory and load it into the instruction register (IR) of the control unit.

1. The MAR is loaded with the contents of the program counter (PC)

2. The memory is interrogated, which results in the next instruction being placed with the memory into the MDR
3. The IR is loaded with the contents of the MDR

or

1. Load the MAR with the contents of the PC, and simultaneously increment the PC
2. Interrogate memory, resulting in the instruction being placed in the MDR
3. Load the IR with the contents of the MDR

### **Decode**

Depending on which output of the decoder is asserted, the remaining 12 bits identify what else is needed to process that instruction

### **Evaluate Address**

asdfasdf  
asdfasdf

### **Fetch Operands**

Loading MAR with the address calculated in the evaluate address phase and reading  
asdfasdf

### **Execute**

Execution of the instruction. In ADD, the addition is performed here in the ALU

### **Store Result**

The result is written to its designated designation. In the case of the ADD instruction, in many computers this action is performed during the EXECUTE phase. Often an ADD instruction can fetch source operands, perform the ADD in the ALU, and store the result in the destination register all in a single clock cycle.

Once the instruction cycle has been completed, the control unit begins the instruction cycle for the next instruction, starting with FETCH. Because the PC was updated during the previous instruction cycle, it contains at this point the address of the instruction stored in the next sequential memory location.

Not all instructions require all six phases.

## Assembly!

Assembly code label

- the first column in an assembly program is used for labels. A label is a way of telling the assembler to remember a specific memory location,
- Labels can be used as positions for branching and jumping to subroutines (BR and JSR) labels can be used like variable names for loading and storing (LD, LDI, LEA, ST, STI)
- the assembler will calculate the proper offsets for you using the label addresses. Only works for PCOffset instructions

```
; -----  
-----  
; Writes "Hello, World" to the console using only system calls. Runs on 64-  
bit Linux only.  
; To assemble and run:  
;  
;     nasm -felf64 hello.asm && ld hello.o && ./a.out  
; -----  
-----  
  
        global    _start  
  
        section   .text  
_start:  mov       rax, 1           ; system call for write  
        mov       rdi, 1           ; file handle 1 is stdout  
        mov       rsi, message     ; address of string to output  
        mov       rdx, 13          ; number of bytes  
        syscall                    ; invoke operating system to do  
the write  
        mov       rax, 60          ; system call for exit  
        xor       rdi, rdi         ; exit code 0  
        syscall                    ; invoke operating system to  
exit  
  
        section   .data  
message: db        "Hello, World", 10 ; note the newline at the end
```

```
.orig x3000
```

## Assembler Directives

| Opcode   | Operand            | Meaning                                                             |
|----------|--------------------|---------------------------------------------------------------------|
| .ORIG    | address            | starting address of the program                                     |
| .END     |                    | end of program                                                      |
| .BLKW    | n                  | allocate n words of storage                                         |
| .FILL    | A                  | allocate one word, initialize with value A                          |
| .STRINGZ | n-character string | allocate n+1 locations, initialize w/characters and null terminator |

## Trap codes

| Code | Equivalent | Description                                 |
|------|------------|---------------------------------------------|
| HALT | TRAP x25   | Halt execution and print message to console |
| IN   |            |                                             |
| OUT  |            | fill in later if relevant                   |
| GETC |            |                                             |
| PUTS |            |                                             |

```
; Example Assembly Program – Add 2 to non-negative number and store into
another memory location
; load number from locations PLACE1,
    .ORIG x3000 ;program starts at address x3000
    LD    R1, PLACE1    ; PLACE is location in memory
                        ; note: offset not specified by programmer
                        ; assembler calculates offset needed
    BRn Done    ;if number is Negative goto end
    ADD    R3, R1, #2    ; Add 2 store into R3
    ST     R3, PLACE2    ; store result into PLACE2
Done HALT          ;halt program
;
PLACE2 .BLKW 1        ; reserve/set aside one word in memory
PLACE1 .FILL x0005    ; initialize number to 5
    .END              ; end of program
```

.fill and .BLKW example program

```
.orig x3000
LEA R6, C
LD R0, A
Halt
A    .fill 65
B    .fill 66
C    .BLKW 5 ; reserve 5 0's into this memory location
D    .fill 68
.end
```

In a table representation...

| Address | Label | HEX  | Command   |
|---------|-------|------|-----------|
| 3000    |       | EC04 | LEA R6, C |
| 3001    |       | 2001 | LD R0, A  |
| 3002    |       | F025 | Halt      |
| 3003    | A     | 0041 | 65        |
| 3004    | B     | 0042 | 66        |
| 3005    | C     | 0000 | 0 1       |
| 3006    |       | 0000 | 0 2       |
| 3007    |       | 0000 | 0 3       |
| 3008    |       | 0000 | 0 4       |
| 3009    |       | 0000 | 0 5       |
| 300A    | D     | 0044 | 68        |

Is assembly easier to do by just making a table?

Example test question: What is the memory address associated with D?

How to save registers

Mult



```

; save registers
; if using r0, r1, r2, save the original values
; don't need to save r0 due to that is where the result of mult will be
stored
ST R1, SAVE_R1    ;save r1
ST R2, SAVE_R2    ;save r2

; do any code here

; restore registers
LD R1, SAVE_R1    ; restore r1 before returning
LD R2, SAVE_R2    ; restore r2 before returning

; only return after restoring the registers
RET

SAVE_R1 .fill 0
SAVE_R2 .fill 0

```

## Get Num

```

.orig x3000
.sub GETNUM; declared subroutine
JSR GETNUM
JSR GETNUM
HALT

GETNUM
; save registers
; program code here
; restore registers
RET

```

- ☐ Don't write too much if not necessary
- ☐ Work on homeworks

## IEEE 16 bit floating point

1 bit sign (0 for positive, 1 for negative)

5 bit exponent (excess-15)

10 bit mantissa (Normalized then remove leading 1)

| S | E | E | E | E | E | M | M | M | M | M | M | M |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | 1 | 0 | 0 | 0 | 1 | 1 | 1 | 0 | 0 | 0 | 0 | 0 |

Equals

(base 2)

### Excess 15

Excess 15 exponent means that the range is shifted so that the true >mathematical value can be negative but the hardware sees a positive number

$$\text{Stored Exponent} = \text{True Exponent} + 15$$

Ex:

True exponent is , the stored value is 15 ()

True exponent is , the stored value is 1 ()

True exponent is , the stored value is 30 ()

Stored values of 0 and 31 are reserved for special cases like zero, subnormal numbers, and infinity/nan

Why Excess?

- Faster comparison
- simple sorting - because the exponent is biased, a more negative exponent always results in a smaller integer
- normalized representation - the bias simplifies the handling of exponents, allowing for smooth handling of both very small (neg exponent) and very large (pos exponent) numbers, whereas 2's complement has asymmetry issues
- handles underflow

How to convert to floating point

1. Write number in normalized binary scientific notation if not already done. The mantissa of this number must have enough bits to fill the mantissa portion of the floating point number (10, or 23)
2. Calculate the characteristic by adding the excess to the exponent
3. Store the sign: 0 = positive, 1 = negative
4. Store the characteristic as calculated above
5. Remove the leading 1 from the mantissa and store

6. (for this class, most quizzes and homework will require writing the value as a hexadecimal value)

### Binary to Decimal Conversion

To convert a 16-bit binary string ( `0 10100 0110000000` ) to decimal:

1. Identify the sign: the first bit is `0` , so the number is positive
2. Calculate the true exponent
  - Stored exponent:
  - True Exponent:
3. Identify the Mantissa:
  - the stored bits are
  - Add the implicit leading 1:
4. Combine and calculate
  - 
  - Shift the binary point 5 places to the right:
  -

### Decimal to Binary Conversion

To convert a decimal number (e.g., **6.5**) to 16-bit binary:

1. **Determine the Sign:** 6.5 is positive, so the sign bit is **0**.
2. **Convert to Binary:** .
3. **Normalize:**
  - Move the decimal point so there is only one `1` to the left: .
  - The True Exponent is **2**.
4. **Calculate Biased Exponent:**
  - .
  - in 5-bit binary is **10001**.
5. **Identify the Mantissa:**
  - Take the bits to the right of the decimal: `101` .
  - Pad with zeros to reach 10 bits: **1010000000**.

6. **Final Result:** 0 10001 1010000000 .

1. Special Numbers Exist
2. Know how to represent zero[]

### Special Numbers

IEEE Defines certain special numbers

- Zero (+ and -)
- Denormalized
- Infinity (+ and -)
- Not a Number (NaN)

Special numbers use patterns of zeros in the characteristic and mantissa field

For this course, you will need to know how to store zero

0 00000 0000000000 is zero

Negative and positive zero are distinct values, though they both compare as equal

+0 = 0x00000000

+0 = 0x0000

-0 = 0x8000

### Infinity

The values +infinity and -infinity are denoted with all ones in the characteristic and all zeroes in the mantissa

The sign bit distinguishes between negative infinity and positive infinity

Like this: 0 11111 0000000000

### NaN

Like this: 0 11111 1000000000

If you divide any number by infinity, you get 0

If you multiply infinity x infinity you get infinity

+/- nonzero div by zero, you get +/- infinity

+/- infinity x zero, you get NaN